# 5343 Advanced Algorithms

Exam 02 Spring 2026

**Instructions:**

- Pencil only. Write all answers on the **detachable answer sheet** (last two pages).
- Use space wisely; align text for maximum readability.
- If you have a question, walk down and ask.
- Phones or other devices are grounds for failure.

| **Section** | **Points** | **Earned** |
|---|---|---|
| I. Multiple Choice ($10 \times 2$) | 20 | |
| II. Fill in the Blank ($5 \times 2$) | 10 | |
| III. Short Answer | 42 | |
| **Total** | **72** | |

---

## Section I Multiple Choice 2 pts each

Circle the best answer on your answer sheet.

[2] 1. Which container guarantees $O(1)$ access to the minimum element?

A. BST

B. Sorted Array

**C. Min-Heap**

D. Hash Table

The heap property keeps the minimum at the root — O(1) access, O(log n) removal.

[2] 2. Binary search requires which combination?

A. Sorted data only

B. O(1) index access only

**C. Sorted data and O(1) index access**

D. A hash function and sorted data

Without O(1) index access, computing `mid` costs O(n) — no faster than linear.

[2] 3. A hash table using linear probing has $\alpha = 0.90$. Expected unsuccessful probes $\approx$:

A. 1.5

B. 2.5

C. 8.5

**D. 50.5**

$\frac{1}{2}(1 + \frac{1}{(1-0.9)^2}) = \frac{1}{2}(101) = 50.5.$

[2] 4. Heapify — building a min-heap from an unsorted array — runs in:

A. $O(\log n)$

B. $O(n \log n)$

**C.** $O(n)$

D. $O(n^2)$

Most nodes are near the bottom and sift down few levels. The sum converges to O(n).

[2] 5. Inorder traversal produces sorted output:

A. On any binary tree

**B. Only on a BST**

C. Only on a complete binary tree

D. Only on a heap

Inorder exploits the BST ordering invariant. Without it, output order is arbitrary.

[2] 6. A hash function $h(s) = \sum \texttt{ASCII}(s_i)$ violates:

A. Determinism

B. Uniformity

**C. Avalanche effect**

D. Speed

"abc", "bca", and "cab" all produce the same sum. One changed character shifts the output by $\leq 1$ — far fewer than 50% of bits.

[2] 7. In a Huffman tree, the highest-frequency symbol gets:

**A. The shortest code — it sits closest to the root**

B. The longest code — to distinguish it from others

C. A code equal in length to all other symbols

D. A code based on alphabetical order

Huffman always merges the two lowest-frequency nodes first, so high-frequency symbols stay near the root and accumulate fewer edge labels.

[2] 8. Worst-case hash table lookup:

A. Chaining: O(1); open addressing: O(n)

B. Chaining: O(log n); open addressing: O(1)

C. Chaining: O(1); open addressing: O(log n)

**D. Both chaining and open addressing: O(n)**

All n keys can hash to one slot (chaining: one chain of n; open addressing: one cluster of n). Expected O(1); worst-case O(n) for both.

[2] 9. The MST of a connected graph is guaranteed unique when:

A. The graph is complete ($K_n$)

**B. All edge weights are distinct**

C. The graph has no cycles

D. Both Prim's and Kruskal's agree on every edge

Distinct weights force the Cut Property to identify exactly one minimum crossing edge per cut — both algorithms are compelled to make the same choices.

[2] 10. A symbol table is queried millions of times per second but updated once per minute. Prefer:

A. Red-Black tree — fewer rotations means faster reads

**B. AVL tree — stricter balance means shorter tree and faster search**

C. Either — they have identical search performance

D. Hash table — O(1) lookup regardless of update rate

AVL height $\leq 1.44 \log n$ vs RB height $\leq 2 \log n$. Fewer comparisons per lookup matters when reads dominate.

---

## Section II Fill in the Blank 2 pts each

Write your answers on the answer sheet, numbered to match.

[2] 11. The load factor of a hash table is $\alpha =$ _____, where $n =$ elements stored, $m =$ table capacity.

$n/m$

[2] 12. Kruskal's algorithm uses _____ to detect whether adding an edge would form a cycle.

Union-Find (Disjoint Set Union / DSU)

[2] 13. When inserting a node into a Red-Black tree, it is initially colored _____.

Red

[2] 14. A _____ tree compresses chains of single-child trie nodes into substring-labeled edges, reducing node count from $O$(total chars) to $O(n)$ keys.

Radix

[2] 15. In double hashing, $h_2(k)$ must never return _____, or the probe sequence revisits the same slot forever.

0 (zero)

---

## Section III Short Answer

*Show your work. Write all answers on the answer sheet.*

[10] 16. **Hash Table — Linear Probing.** Table size $m = 11$, hash function $h(k) = k \bmod 11$. Insert keys **22, 35, 15, 44, 55, 26** in order.

[6] (a) For each key, record $h(k)$, the probe sequence, and the final slot. Use the table on the answer sheet.

[2] (b) Draw the final table state on the answer sheet (slots 0–10; mark empty with `---`).

[2] (c) Name the clustering phenomenon visible in this trace and explain it in one sentence.

### Solution

**(a)**

| Key | $h(k)$ | Probe sequence | Final slot |
|---|---|---|---|
| 22 | 0 | 0 (empty) | 0 |
| 35 | 2 | 2 (empty) | 2 |
| 15 | 4 | 4 (empty) | 4 |
| 44 | 0 | 0(22)→1(empty) | 1 |
| 55 | 0 | 0(22)→1(44)→2(35)→3(empty) | 3 |
| 26 | 4 | 4(15)→5(empty) | 5 |

**(b)**

| 0 | 1 | 2 | 3 | 4 | 5 | 6-7 |
|---|---|---|---|---|---|---|
| 22 | 44 | 35 | 55 | 15 | 26 | — |

**(c)**

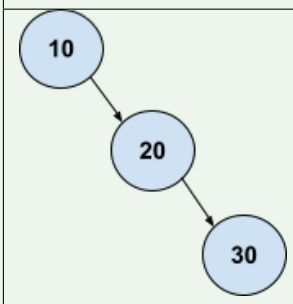
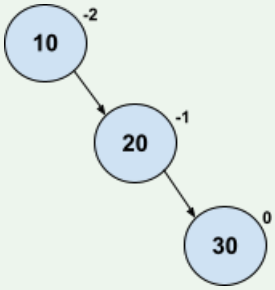
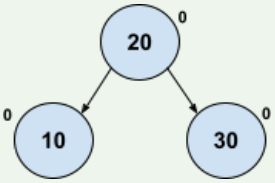
**Primary clustering**: keys hashing to different initial slots (0, 2, 4) pile into the same contiguous run (slots 0–5) because each collision extends the occupied block, forcing subsequent insertions to probe further.

[12] 17. **AVL Tree — Insertions and Rotations.** Insert in order: 10, 20, 30, 15, 5, 25, 28.

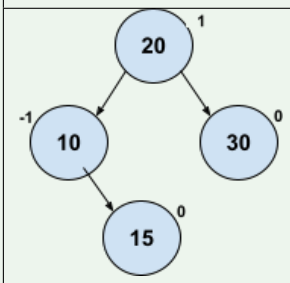
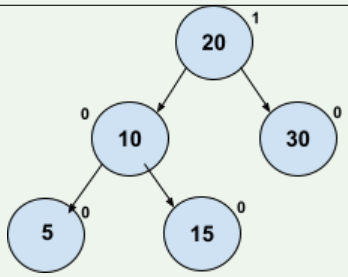
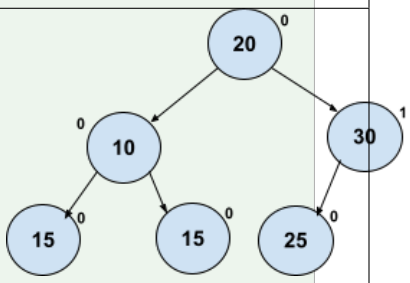
*(a) After inserting 10, 20, 30 the tree is unbalanced. Name the imbalanced node, the rotation case, and draw the tree after the fix* (4 pts)

**Solution**

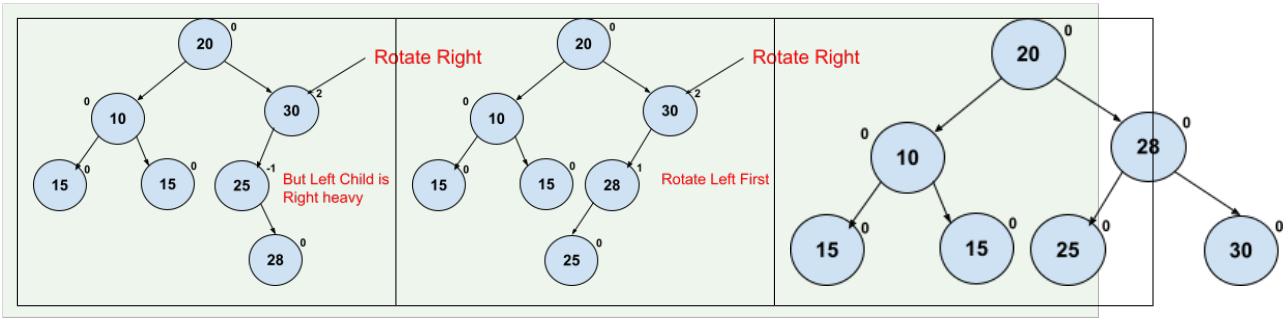
No rotations until all values inserted.

| 10,20,30 Inserted | Balance Factors | Left Rotate at 20 |
|---|---|---|
|  |  |  |

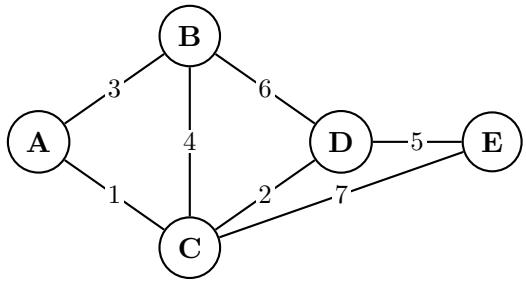
*(b) After inserting 15, 5, 25 — tree with balance factors* (4 pts)

| Add 15 | Add 5 | Add 25 |
|---|---|---|
|  |  |  |

*(c) After inserting 28 — rotation case, steps, final tree* (4 pts)



[10] 18. **Minimum Spanning Tree — Kruskal's Algorithm.**




[6] (a) Trace Kruskal's. Complete the table on the answer sheet (edges in sorted order, action, components after).

[2] (b) State the MST edges and total weight.

[2] (c) Why is the B–C edge skipped? Explain using Union-Find terminology.

## Solution

Sorted: A–C(1), C–D(2), A–B(3), B–C(4), D–E(5), B–D(6), C–E(7)

**(a)**

| Step | Edge | Wt | Action | Components |
|---|---|---|---|---|
| 1 | A–C | 1 | Add | {A,C}{B}{D}{E} |
| 2 | C–D | 2 | Add | {A,C,D}{B}{E} |
| 3 | A–B | 3 | Add | {A,B,C,D}{E} |
| 4 | B–C | 4 | Skip | {A,B,C,D}{E} |
| 5 | D–E | 5 | Add | {A,B,C,D,E} |
| 6 | B–D | 6 | Skip | {A,B,C,D,E} |
| 7 | C–E | 7 | Skip | {A,B,C,D,E} |

**(b)**

MST: A–C(1), C–D(2), A–B(3), D–E(5). Weight = **11**.

**(c)**

`find(B)==find(C)` — both in {A,B,C,D}. Adding the edge would close a cycle.

[10] 19. **Huffman Tree — Build and Encode.**

| Char | Freq | Char | Freq |
|---|---|---|---|
| A | 12 | D | 4 |
| B | 7 | E | 3 |
| C | 5 | F | 2 |

Total: 33

[3] (a) Trace the 5 min-heap merge steps on the answer sheet (left node, right node, new weight).

[3] (b) Draw the resulting tree on the answer sheet (0 = left edge, 1 = right edge).

[4] (c) Complete the encoding table on the answer sheet and compute: total Huffman bits, total fixed 3-bit bits, and bits saved.

## Solution

**(a)**

Initial: F(2), E(3), D(4), C(5), B(7), A(12)

| Step | Left Weight | Right Weight | New Weight |
|---|---|---|---|
| 1 | F(2) | E(3) | 5 |
| 2 | D(4) | C(5) | 9 |
| 3 | FE(5) | B(7) | 12 |
| 4 | CD(9) | A(12) | 21 |
| 5 | FEB(12) | ACD(21) | 33 |

**(b)**

Root(33): left FEB(12), right DCA(21).
FEB: left FE(5)→left F(2)/right E(3); right B(7).
DCA: left DC(9)→left D(4)/right C(5); right A(12).

**(c)**

| Char | Freq | Code | Len | Bits |
|---|---|---|---|---|
| A | 12 | 11 | 2 | 24 |
| B | 7 | 01 | 2 | 14 |
| C | 5 | 101 | 3 | 15 |
| D | 4 | 100 | 3 | 12 |
| E | 3 | 001 | 3 | 9 |
| F | 2 | 000 | 3 | 6 |
| | | | Huffman total: | 80 |
| | | | Fixed 3-bit ($33 \times 3$): | 99 |
| | | | Saved: | 19 |

# Answer Sheet — Exam 02

5343 Advanced Algorithms  Spring 2026

Name: _____  Total: _____ / 72

---

## Section I — Multiple Choice

*2 pts each / 20 total*

| Q | A | B | C | D | Pts |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |
| 5 | | | | | |
| 6 | | | | | |
| 7 | | | | | |
| 8 | | | | | |
| 9 | | | | | |
| 10 | | | | | |

## Section II — Fill in the Blank

*2 pts each / 10 total*

**1.** _____ /2

**2.** _____ /2

**3.** _____ /2

**4.** _____ /2

**5.** _____ /2

## Section III — Short Answer */42 total*

### SA1 — Hash Table Trace */10*

*(a) Probe trace* *(6 pts)*
*(b) Final table state* *(2 pts)*
*(c) Clustering phenomenon* *(2 pts)*

**(a)**

| Key | $h(k)$ | Probe sequence | Final slot |
|---|---|---|---|
| 22 | 0 | 0 (empty) | 0 |
| 35 | 2 | 2 (empty) | 2 |
| 15 | 4 | 4 (empty) | 4 |
| 44 | 0 | 0(22)→1(empty) | 1 |
| 55 | 0 | 0(22)→1(44)→2(35)→3(empty) | 3 |
| 26 | 4 | 4(15)→5(empty) | 5 |

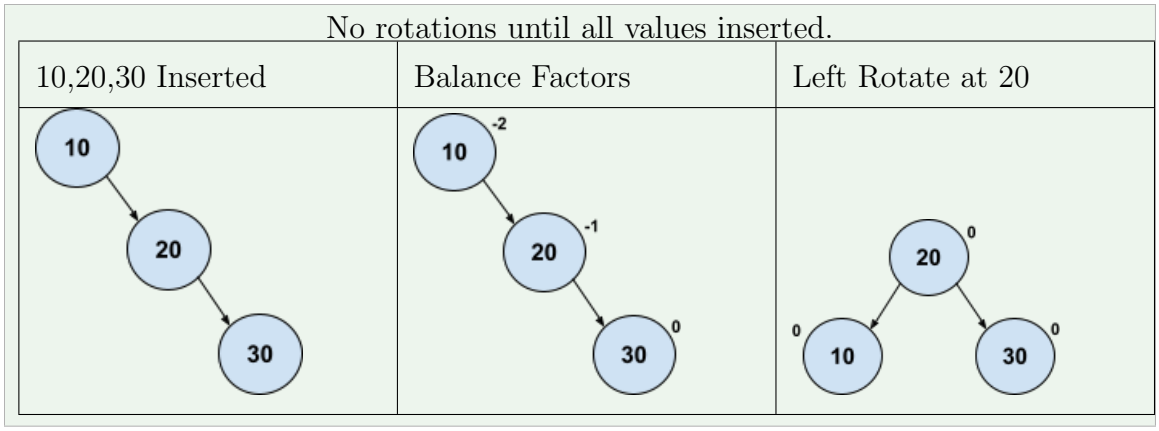
**(b)** Slot 0:22, 1:44, 2:35, 3:55, 4:15, 5:26, 6–10: —

| 0 | 1 | 2 | 3 | 4 | 5 | 6-7 |
|---|---|---|---|---|---|---|
| 22 | 44 | 35 | 55 | 15 | 26 | — |

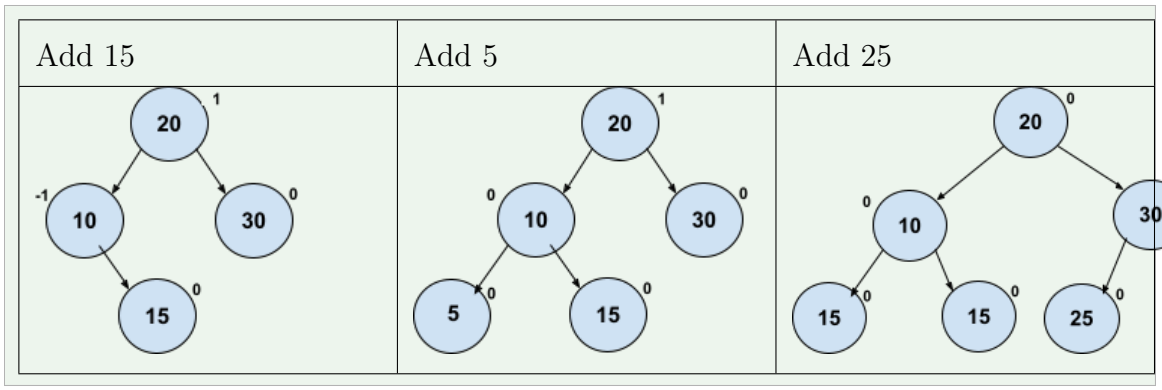
**(c) Primary clustering**: keys hashing to different initial slots (0, 2, 4) pile into the same contiguous run (slots 0–5) because each collision extends the occupied block, forcing subsequent insertions to probe further.

## SA2 — AVL Rotations /12

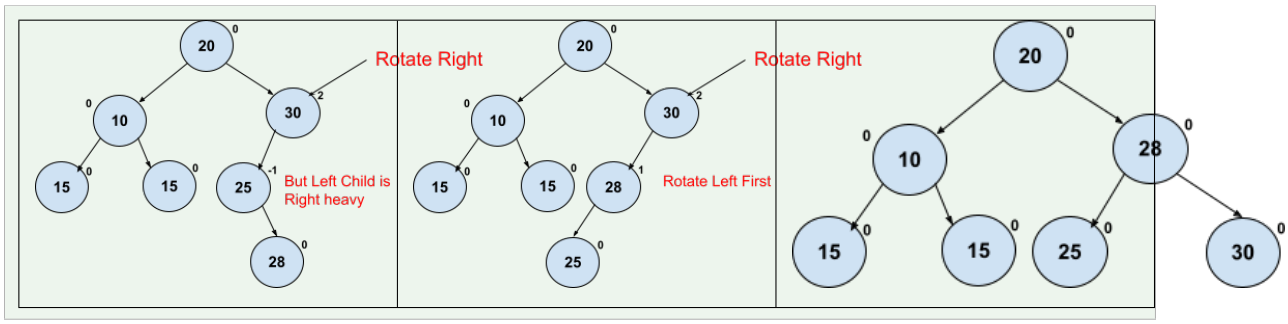
*(a) After inserting 10, 20, 30 — rotation case and result tree* (4 pts)




*(b) After inserting 15, 5, 25 — tree with balance factors* (4 pts)




*(c) After inserting 28 — rotation case, steps, final tree* (4 pts)



# Answer Sheet — Exam 02, Page 2

Name: _____

**SA3 — Kruskal's Algorithm** */10*

*(a) Trace table* (6 pts)

| Step | Edge | Wt | Action | Components after |
|---|---|---|---|---|
| 1 | | | | |
| 2 | | | | |
| 3 | | | | |
| 4 | | | | |
| 5 | | | | |
| 6 | | | | |
| 7 | | | | |

*(b) MST edges and total weight* (2 pts)

*(c) Why B–C is skipped (Union-Find)* (2 pts)

**SA4 — Huffman Tree** */10*

*(a) Min-heap merge trace* (3 pts)

| **Step** | **Left (weight)** | **Right (weight)** | **New weight** |
|---|---|---|---|
| 1 | | | |
| 2 | | | |
| 3 | | | |
| 4 | | | |
| 5 | | | |

*(b) Huffman tree sketch (0 = left edge, 1 = right edge)* (3 pts)

*(c) Encoding table and bit totals* (4 pts)

| **Char** | **Freq** | **Code** | **Length** | **Bits (F×L)** |
|---|---|---|---|---|
| A | 12 | | | |
| B | 7 | | | |
| C | 5 | | | |
| D | 4 | | | |
| E | 3 | | | |
| F | 2 | | | |
| | | | **Total Huffman bits:** | |
| | | | **Fixed 3-bit** $(33 \times 3)$**:** | 99 |
| | | | **Bits saved:** | |